QUALIDADE DE SOFTWARE · UFC · 2026.2

Refatoração de test smells no LightRAG

Detecção com TEMPY, refatoração guiada pelo CATSPY e contribuição para o projeto original (PR #4290)

Pedro Rivaldo Tabosa Silva · matrícula 511881

github.com/HKUDS/LightRAG

ROTEIRO

O que vou mostrar

01

O projeto

LightRAG e o processo que segui

02

Smells encontrados

O que o TEMPY detectou e o que escolhi

03

Refatorações

25 ocorrências e exemplos de antes e depois

04

Métricas

Radon, pytest e TEMPY, antes e depois

05

Contribuição

O Pull Request #4290

06

Reflexões

Dificuldades, CATSPY e lições

01 · O PROJETO

LightRAG: RAG com grafos de conhecimento

Framework open source em Python do **HKU Data Intelligence Lab**. Extrai entidades e relações dos documentos, monta um grafo e o combina com busca vetorial para responder perguntas.

Tem servidor **FastAPI** com interface web e suporta vários bancos (PostgreSQL, Neo4j, Redis, Milvus...) e provedores de LLM.

Testes em **pytest**, na pasta **tests/**, que espelha o pacote; marcados como **offline** (gate do CI) ou **integration** (pulados por padrão).

537
arquivos de teste analisados

9.142
testes coletados pelo pytest

76,09%
de cobertura (linhas + branches)

O processo, em cinco passos

1

Fork e commit base

Branch criada a partir do commit da planilha: 453dce83d

2

Medir antes

Radon, pytest com cobertura e TEMPY, com os scripts da disciplina

3

Refatorar

Uma ocorrência por commit, consultando o CATSPY e registrando no diário

4

Medir depois

Mesmos scripts, no último commit de refatoração

5

Contribuir

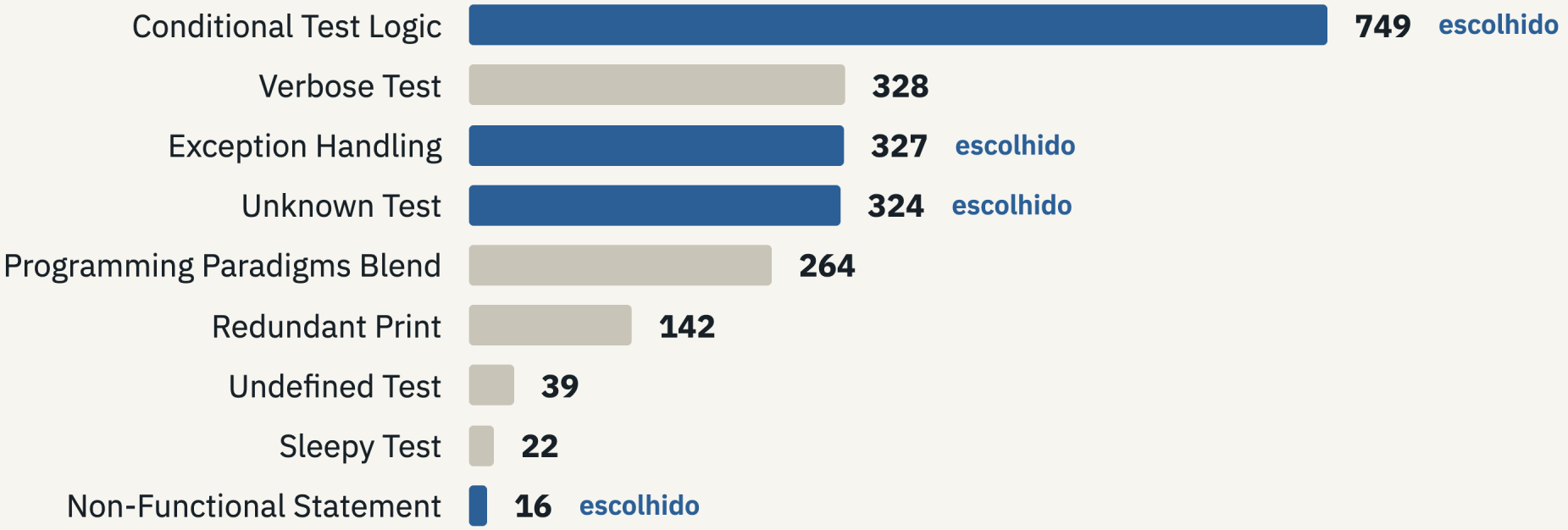
Pull Request #4290 para o repositório original

Cada refatoração foi validada na hora: **pytest** no arquivo, **ruff** e **TEMPY** no modo **--arquivo**.

02 · SMELLS ENCONTRADOS

TEMPY no estado inicial

2.211
test smells em 537 arquivos



Fonte: metrics-before-tempy/resumo.json · commit 453dce83d

02 · SMELLS ENCONTRADOS

Nem tudo que o TEMPY aponta é smell

304 de 327

ocorrências de **Exception Handling** eram blocos **try/finally** de limpeza (fechar storages), não tratamento manual de exceção

245 de 324

ocorrências de **Unknown Test** já verificavam com **pytest.raises**, que o TEMPY não reconhece como asserção

Por isso, **cada ocorrência foi conferida à mão** antes de entrar na lista de refatorações.

Quais tipos escolhi, e por quê

Tipo	Severidade	Total	Decisão
Exception Handling	Alta	327	Escolhido: 6 refatorações
Conditional Test Logic	Alta	749	Escolhido: 9 refatorações
Unknown Test	Alta	324	Escolhido: 6 refatorações
Non-Functional Statement	Baixa	16	Escolhido: 4 refatorações
Verbose Test	Alta	328	Fora: mudança grande e arriscada
Undefined Test	Alta	39	Fora: são funções auxiliares
Sleepy Test	Média	22	Fora: a espera faz parte do cenário
Paradigms Blend	Média	264	Fora: mudança estrutural no módulo
Redundant Print	Baixa	142	Fora: só em testes de integração

03 · REFATORAÇÕES

25 refatorações, uma por commit

6	9	6	4
Exception Handling	Conditional Test Logic	Unknown Test	Non-Functional Statement

+10 de quebra

Algumas refatorações removeram outros smells junto: 9 Conditional Test Logic e 1 Non-Functional Statement.

0 smells novos

Comparei tipo e método no smells.csv antes e depois, nos 18 arquivos alterados.

Lista completa na seção 3.2 do relatório (R01 a R25)

EXEMPLO R01 · EXCEPTION HANDLING

try/except manual vira pytest.raises

ANTES

```
try:
    json.loads('{\n "x": "abc')
except json.JSONDecodeError as exc:
    result = create_prefixed_exception(
        exc, "`entity`")
assert isinstance(result, RuntimeError)
assert "JSONDecodeError" in str(result)
```

DEPOIS

```
with pytest.raises(json.JSONDecodeError) as exc_info:
    json.loads('{\n "x": "abc')
result = create_prefixed_exception(
    exc_info.value, "`entity`")
assert isinstance(result, RuntimeError)
assert "JSONDecodeError" in str(result)
```

Por que era smell: o teste capturava a exceção à mão. Se o json.loads deixasse de lançá-la, a falha seria um NameError confuso; agora o pytest diz “DID NOT RAISE”.

EXEMPLO R06 · EXCEPTION HANDLING

Quando a receita do catálogo não serve

O CATSPY SUGERE

Trocar o try/except por pytest.raises

MAS NO PROJETO

O json_repair não lança exceção nessa entrada: devolve uma lista

SOLUÇÃO

Extract Method: o parser antigo vira uma função auxiliar, e o teste fica só com as asserções

```
def _legacy_greedy_load(raw: str) → dict:
    m = re.search(r"\{[\s\S]*\}", raw)
    if m is None:
        return {}
    try:
        obj = json_repair.loads(m.group(0))
    except Exception:
        return {}
    return obj if isinstance(obj, dict) else {}

def test_greedy_regex_would_fail_same_input():
    raw = '{"facts":[{"text":"ok"}]} trailing {brace}'
    assert _legacy_greedy_load(raw) == {}
    assert tolerant_load_json_dict(raw) == {...}
```

EXEMPLO R12 E R13 · CONDITIONAL TEST LOGIC

Laços dentro do teste viram parametrize

ANTES · 1 TESTE, 2 LAÇOS

```
def test_env_bool_truthy_falsy(monkeypatch):
    for raw in ("1", "true", "yes", "ON"):
        monkeypatch.setenv("X", raw)
        assert env_bool("X", False) is True
    for raw in ("0", "false", "no", "off"):
        monkeypatch.setenv("X", raw)
        assert env_bool("X", True) is False
```

DEPOIS · 2 TESTES, 8 CASOS

```
@pytest.mark.parametrize(
    "raw", ["1", "true", "yes", "ON"])
def test_env_bool_truthy(monkeypatch, raw):
    monkeypatch.setenv("X", raw)
    assert env_bool("X", False) is True

@pytest.mark.parametrize(
    "raw", ["0", "false", "no", "off"])
def test_env_bool_falsy(monkeypatch, raw):
    monkeypatch.setenv("X", raw)
    assert env_bool("X", True) is False
```

Ganho: antes, a primeira falha escondia as outras; agora cada valor é um caso e a falha aponta o valor exato.

tests/parser/external/test_common.py · commits b155b63bb e 2536721da · trecho simplificado

EXEMPLO R20 · UNKNOWN TEST

Um teste sem nenhuma asserção

ANTES

```
def _explode(*args, **kwargs):
    raise AssertionError("opened the zip")

monkeypatch.setattr(zipfile, "ZipFile", _explode)
NativeDocxParser().validate_source(src, "ok.docx")
# nenhuma asserção
```

DEPOIS

```
opened: list[tuple] = []

def _explode(*args, **kwargs):
    opened.append(args)
    raise RuntimeError("opened the zip")

monkeypatch.setattr(zipfile, "ZipFile", _explode)
NativeDocxParser().validate_source(src, "ok.docx")
assert opened == []
```

Risco escondido: AssertionError é uma Exception. Um **except Exception** no código engoliria a regressão e o teste continuaria passando.

tests/parser/docx/test_zip_budget.py · commit e1bb5d1b7 · trecho simplificado

EXEMPLOS R22 A R25 · NON-FUNCTIONAL STATEMENT

Trocando o pass por algo que diz alguma coisa

R22: pass dentro do with vira a verificação que faltava

```
with _workspace_env_isolated("REDIS_WORKSPACE"):
    pass
assert "REDIS_WORKSPACE" not in os.environ
```

```
with _workspace_env_isolated("REDIS_WORKSPACE"):
    assert "REDIS_WORKSPACE" not in os.environ
assert "REDIS_WORKSPACE" not in os.environ
```

R23 a R25: pass no corpo de classes dublê vira uma docstring

```
class _ResumePurged(Exception):
    pass
```

```
class _ResumePurged(Exception):
    """Raised after the real purge ran..."""
```

À esquerda, antes; à direita, depois · commits cdc915b71, 24feebf30, 032079db6, d247c44ad

Antes e depois, lado a lado

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	2.211	2.176	−35
Testes aprovados	8.889	8.916	+27
Testes com falha	7	7	0
Cobertura total (%)	76,09	76,09	0,00 p.p.
SLOC dos testes	157.827	157.828	+1
CC média das funções de teste	2,898	2,896	−0,002
Manutenibilidade (MI) média dos arquivos de teste	56,829	56,838	+0,009

Fontes: pastas metrics-before-* e metrics-after-* (Radon, pytest, TEMPY)

04 · MÉTRICAS

Como ler esses números

−35

smells, queda de 1,6%

Queda nos quatro tipos trabalhados: Conditional Test Logic −18, Exception Handling −6, Unknown Test −6, Non-Functional Statement −5.

+27

testes, sem cobertura nova

É o efeito do parametrize: cada valor virou um caso. Cobertura igual, as mesmas 7 falhas de ambiente e os mesmos 246 ignorados.

1 → 2–4

CC onde entrou assert

O Radon conta cada assert como ponto de decisão. A métrica piorou justamente porque o teste passou a verificar algo.

As médias quase não mudam porque são **12.805 funções de teste** em 599 arquivos, e eu alterei 18.

05 · CONTRIBUIÇÃO

O Pull Request para o projeto original

#4290

HKUDS/LightRAG · branch main

Aguardando resposta

- ✓ Branch própria a partir do main atual (14 commits à frente); cherry-pick dos 25 commits sem conflitos
- ✓ Só 18 arquivos de teste (+130 / −114 linhas), nenhum código de produção
- ✓ 472 testes dos arquivos alterados passando e todos os hooks de pre-commit ok
- ✓ Descrição no template do projeto, com as mudanças agrupadas por tipo de smell
- ✓ Métricas e scripts ficaram fora do PR: foram para o fork depois (commit 2f7a5f5e7)

Dificuldades

Ambiente

O projeto não tem o extra [dev] do tutorial: instalei os extras do CI e, à parte, radon, pytest-html e pytest-cov, que um novo uv sync remove.

Execução

Sete testes falham no macOS por causa do ambiente, e precisei confirmar que não tinham relação com o meu trabalho.

Entendimento do código

Projeto grande e cheio de regras de domínio. Escrever asserções com sentido exigiu ler a implementação das funções testadas.

Ferramentas

O TEMPY usa heurísticas por linha e gera muitos falsos positivos. Filtrar as ocorrências genuínas tomou boa parte do tempo.

06 · REFLEXÕES

Minha opinião sobre o CATSPY

Como usei

- Definição, para decidir se a ocorrência era genuína
- Exemplo refatorado, como ponto de partida
- Severidade, para priorizar os tipos
- Efeitos negativos, para justificar o PR

Onde não se aplicou

- Exemplos em unittest: traduzi para pytest.raises e parametrize
- R06: o código não lança exceção
- R02, R03, R05: exceções dentro de threads
- R23 a R25: pass em classes dublê

O que melhoraria

- Exemplos em pytest: raises, parametrize, fixtures
- Orientação sobre falsos positivos e quando o smell é aceitável
- Casos de testes concorrentes e de classes dublê

O que aprendi

1. Ferramenta de detecção é **ponto de partida, não veredito**: cada ocorrência precisa de julgamento humano.
2. Um teste que “só não pode lançar exceção” **quase sempre pode afirmar algo melhor**.
3. Vale **provar que o teste ainda falha**, simulando o defeito, como fiz na R04.
4. Métrica pior nem sempre é código pior, e **passos pequenos** facilitam revisão e rastreio.

Obrigado!

PR: github.com/HKUDS/LightRAG/pull/4290
Fork: github.com/pedrorivald/LightRAG